



ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

ERICK FERREIRA MENDES

PABLO PEREIRA

SISTEMA DE CONTROLE DE ESTACIONAMENTO ROTATIVO

BRASÍLIA

2026

ERICK FERREIRA MENDES

PABLO PEREIRA

SISTEMA DE CONTROLE DE ESTACIONAMENTO ROTATIVO

Trabalho de avaliação apresentado ao curso de
Análise e desenvolvimento de Sistemas, da
Faculdade Brasília.

Orientador: WILLIANS LUIZ GOMES

BRASÍLIA

2026

Sumário

1. INTRODUÇÃO	5
1.1 Objetivo do projeto.....	5
1.2 Escopo	5
1.3 Definições, siglas e abreviações	6
2. ANÁLISE DO PROBLEMA.....	8
2.1 Contexto do sistema	8
2.2 Stakeholders (partes interessadas).....	8
2.3 Levantamento inicial	9
3. ANÁLISE DE REQUISITOS	10
3.1 Requisitos funcionais.....	10
3.2 Requisitos não funcionais.....	11
3.3 Regras de negócio.....	12
3.4 Casos de uso (descrição textual)	13
4. MODELAGEM DO SISTEMA.....	16
4.1 DFD (Diagrama de Fluxo de Dados).....	16
4.1 Diagramas UML.....	16
4.1.1 Diagrama de Casos de Uso	16
4.1.2 Diagrama de Classes	17
4.1.3 Diagrama de Sequência	18
5. PROTOTIPAÇÃO.....	18
5.1 Prototipação no Figma	18
5.2 Descrição das telas (Descrição das interfaces)	19
6. ARQUITETURA DO SISTEMA.....	19
6.1 Visão Geral da Arquitetura	19
6.2 Tecnologias utilizadas.....	20
6.3 Diagrama de arquitetura.....	21
7. PROJETO DE BANCO DE DADOS	22
7.1 Modelo Conceitual.....	22
7.2 Modelo Lógico	22
7.3 Modelo Físico	23
8. DESENVOLVIMENTO / PROJETO FUNCIONAL	23
8.1 Código da aplicação (documentado).....	23
8.2 Scripts SQL (documentados)	24
8.3 Padrões de codificação utilizados	24

9. PLANO DE TESTES (item importante adicionado)	25
9.1 Estratégia de testes	25
9.2 Casos de teste	26
9.3 Resultados dos testes	26
10. CRONOGRAMA DO PROJETO	27
10.1 Etapas	27
10.2 Prazos	27
10.3 Marcos (milestones)	28
11. ORÇAMENTO	28
11.1 Custos estimados	28
11.2 Recursos necessários	29
12. IMPLANTAÇÃO E MANUTENÇÃO (item adicionado)	29
13. REFERÊNCIAS BIBLIOGRÁFICAS	30
14. ANEXOS	30
14.1 Links úteis	30
14.2 Protótipos	30
15. LINK DO REPOSITÓRIO	31
15.1 GitHub do projeto	31

1. INTRODUÇÃO

1.1 Objetivo do projeto

O objetivo principal deste projeto é desenvolver um Sistema de Controle de Estacionamento Rotativo do tipo Desktop, capaz de gerenciar e automatizar o fluxo de entrada e saída de veículos. O software visa substituir controles manuais e analógicos (como anotações em papel ou planilhas genéricas), mitigando erros humanos no cálculo de tempo de estadia e evitando a perda de informações do fluxo de caixa.

Secundariamente, como objetivo acadêmico, o projeto busca aplicar na prática os conceitos da disciplina de Análise e Modelagem de Sistemas, englobando o levantamento de requisitos, a modelagem de dados e a implementação arquitetural estruturada, utilizando os pilares da Programação Orientada a Objetos (POO).

1.2 Escopo

Para garantir a coesão e o sucesso da entrega, é necessário delimitar as fronteiras do sistema.

Fazem parte do escopo deste projeto:

- Cadastro de entrada de veículos, armazenando placa, categoria (Carro ou Moto) e o carimbo de data/hora do sistema.
- Painel de monitoramento em tempo real dos veículos atualmente estacionados no pátio.
- Cálculo matemático automatizado para encerramento de estadias, considerando tempo total, tipo de veículo, regra de tolerância de isenção e valor da fração por hora.

- Geração de recibo virtual (em tela) para a conferência e finalização do atendimento.
- Módulo de consulta de histórico financeiro, listando todas as movimentações concluídas e os valores arrecadado.

NÃO fazem parte do escopo (Restrições e Limitações):

- Gestão de clientes mensalistas, credenciados ou convênios de descontos.
- Controle de múltiplas filiais de estacionamentos (o sistema é projetado para operação local e individual).
- Módulo de autenticação de usuários (Login/Senha) e níveis de acesso, presumindo-se que a guarita possui acesso único.
- Integração física com hardwares externos, como cancelas automáticas, sensores de presença de vagas, leitores de código de barras ou impressoras térmicas de papel.
- Reserva de vagas de forma online ou via aplicativo móvel.

1.3 Definições, siglas e abreviações

Para a correta compreensão deste documento e da modelagem arquitetural subsequente, definem-se os seguintes termos e siglas:

- DAO (Data Access Object): Padrão de projeto de software utilizado na arquitetura deste sistema para isolar e encapsular os comandos de acesso e persistência no banco de dados.
- POO: Programação Orientada a Objetos. Paradigma de desenvolvimento utilizado para modelar o domínio do problema através de classes, herança e polimorfismo (ex: diferenciação de cálculo entre Carro e Moto).
- SGBD: Sistema de Gerenciamento de Banco de Dados. Software responsável por armazenar os dados do sistema, sendo o MySQL a tecnologia escolhida para este projeto.

- Swing: Biblioteca nativa da linguagem Java utilizada para a construção e renderização da Interface Gráfica do Usuário (UI) em ambientes Desktop.
- Ticket / Movimentação: Entidade principal do sistema que representa o ciclo de vida completo da estadia de um veículo, desde a sua admissão (status "ATIVO") até o seu pagamento (status "CONCLUIDO").
- UI (User Interface): Interface de Usuário. Refere-se às telas, botões e tabelas com as quais o atendente interage fisicamente.

2. ANÁLISE DO PROBLEMA

2.1 Contexto do sistema

O Sistema de Controle de Estacionamento Rotativo está inserido no contexto operacional diário de pequenos a médios estacionamentos privados. O ambiente de implantação caracteriza-se por guaritas ou pontos de cobrança locais, onde a agilidade no atendimento é fundamental para evitar a formação de filas.

O problema central abordado por este projeto é a ineficiência e a vulnerabilidade dos controles analógicos. Atualmente, estabelecimentos de menor porte dependem da emissão de tickets em papel e de cálculos manuais realizados pelos atendentes. Esse cenário gera margem para falhas humanas (como a cobrança incorreta de tarifas), extravio de registros, ausência de transparência nos ganhos diários e lentidão na liberação dos veículos. O software desenvolvido atua diretamente na automação dessas rotinas, trazendo velocidade e integridade matemática para a operação.

2.2 Stakeholders (partes interessadas)

Os stakeholders representam as pessoas ou entidades que afetam ou são afetadas pelo desenvolvimento e implementação do sistema. Para este domínio, identificam-se três grupos principais:

- **Proprietário / Administrador:** O principal interessado no aspecto gerencial e financeiro. Busca a eliminação de desvios de caixa, a precisão no cálculo das horas e a capacidade de consultar o faturamento através de um histórico confiável.
- **Operador de Caixa / Atendente:** O usuário direto e contínuo do sistema. Necessita de uma interface gráfica intuitiva, rápida e livre de

complexidades, que automatize o cálculo do troco ou do valor devido sem a necessidade de intervenção manual em calculadoras.

- Cliente (Motorista): Parte interessada indireta. Embora não opere o sistema, é beneficiado pela agilidade no atendimento (redução de filas na saída), pela garantia de uma cobrança justa (baseada no cálculo exato da máquina) e pela emissão de um recibo claro contendo o seu tempo de permanência.

2.3 Levantamento inicial

O levantamento inicial de requisitos foi conduzido com base na observação das rotinas padrão de controle de pátios de veículos. Identificou-se que a regra de negócio central gravita em torno do controle de estado de uma entidade única: a placa do veículo associada ao momento exato da sua chegada.

Durante a concepção, foi feito como premissa a criação de um sistema de arquitetura local (Desktop), visando eliminar a dependência de conexões de internet (frequentemente instáveis em guaritas) e talvez reduzir os custos de infraestrutura em nuvem para o proprietário.

A modelagem preliminar definiu a necessidade de três operações fundamentais para a viabilidade do produto mínimo viável (MVP):

- Registro de entrada com captura automática do timestamp (data/hora) do sistema.
- Cálculo de estadia automatizado com suporte a variações de tarifas (polimorfismo entre Carro e Moto) e aplicação de regras de tolerância.
- Persistência permanente de dados para auditoria (fechamento de registros) utilizando um banco de dados relacional leve.

3. ANÁLISE DE REQUISITOS

3.1 Requisitos funcionais

O que o programa tem que fazer:

- RF01 - Registrar Entrada: O sistema deve permitir o registro da entrada de veículos no pátio, salvando a placa, o tipo do veículo (Carro ou Moto) e o horário exato da admissão.
- RF02 - Validar Placa: O sistema deve validar os dados inseridos na entrada, impedindo o cadastro de placas em branco ou que possuam menos de 7 caracteres.
- RF03 - Bloquear Duplicidade: O sistema não deve permitir o registro de uma nova entrada para um veículo que já conste com o status ativo no pátio.
- RF04 - Registrar Saída: O sistema deve permitir a localização de um veículo estacionado por meio de sua placa para efetuar a saída.
- RF05 - Calcular Tarifa: O sistema deve calcular automaticamente o valor devido no momento da saída, utilizando o tempo total de permanência no estacionamento.
- RF06 - Aplicar Tolerância: O sistema deve conceder isenção de pagamento (R\$ 0,00) para veículos cujo tempo de permanência seja menor ou igual a 15 minutos.
- RF07 - Precificação Dinâmica: O sistema deve realizar a cobrança por hora cheia ou iniciada, aplicando a tarifa de R\$ 10,00 para carros e R\$ 5,00 para motos.
- RF08 - Emitir Recibo de Conferência: O sistema deve exibir um resumo financeiro e de tempo (recibo) na tela para confirmação do atendente antes de registrar o pagamento e liberar o veículo.
- RF09 - Monitorar Pátio: O sistema deve exibir, em tempo real, uma listagem de todos os veículos que estão ocupando o estacionamento no momento.

- RF10 - Consultar Histórico: O sistema deve manter e disponibilizar uma tabela contendo o histórico de todas as movimentações concluídas, incluindo horários de entrada, saída e o valor arrecadado.

3.2 Requisitos não funcionais

- RNF01 - Plataforma e Interface: O software deve ser uma aplicação do tipo Desktop, desenvolvida nativamente na linguagem Java e utilizando a biblioteca Swing para a renderização da interface gráfica do usuário.
- RNF02 - Persistência de Dados: O sistema deve utilizar o Sistema de Gerenciamento de Banco de Dados (SGBD) relacional MySQL para o armazenamento das movimentações financeiras e de veículos.
- RNF03 - Arquitetura de Software: O código-fonte deve ser estruturado em um padrão arquitetural de camadas, separando nitidamente o Modelo de Negócios (Classes Base), o Acesso a Dados (DAO) e a Interface (UI).
- RNF04 - Paradigma de Desenvolvimento: O projeto deve aplicar estritamente os pilares da Programação Orientada a Objetos, garantindo o uso de classes abstratas, herança e métodos polimórficos para o cálculo das tarifas.
- RNF05 - Disponibilidade (Offline): O sistema deve ser capaz de operar de forma 100% autônoma e local, sem a necessidade de requisições web, conexão à internet ou consumo de APIs de terceiros.
- RNF06 - Precisão Temporal: O sistema deve garantir a precisão no cálculo das estadias utilizando os pacotes de manipulação de data e hora locais da linguagem (java.time) mapeados diretamente para o tipo DATETIME do banco de dados.

3.3 Regras de negócio

RN01 - Tabela de Tarifação por Categoria: O valor da hora de estacionamento é fixo e difere de acordo com a categoria do veículo registrada na admissão. Veículos da categoria "CARRO" possuem a tarifa de R\$ 10,00 por hora, enquanto veículos da categoria "MOTO" possuem a tarifa de R\$ 5,00 por hora.

RN02 - Período de Tolerância Gratuita: Todo veículo, independente de sua categoria, possui o direito a uma tolerância de isenção tarifária caso o seu tempo total de permanência no pátio seja menor ou igual a 15 minutos (0 a 15 minutos). O valor gerado para cobrança nesses casos será obrigatoriamente R\$ 0,00.

RN03 - Fração de Cobrança por Hora Cheia: Após ultrapassado o período de tolerância estipulado na RN02, a cobrança não será fracionada por minuto, mas sim por hora iniciada (arredondamento para o teto matemático).

- Exemplo prático: 16 minutos a 60 minutos geram a cobrança de 1 hora. 61 minutos a 120 minutos geram a cobrança de 2 horas, e assim sucessivamente.

RN04 - Unicidade de Veículo Ativo: Um mesmo veículo (identificado unicamente pela sua placa) não pode ter mais de um registro em aberto simultaneamente no estacionamento. O sistema deve validar no banco de dados se a placa já possui o status "ATIVO" e impedir uma nova entrada até que a saída do registro anterior seja processada.

RN05 - Integridade da Identificação (Placa): Para fins de segurança e consistência de dados, o registro de entrada só será efetivado se a string informada no campo "Placa" contiver, no mínimo, 7 caracteres, abrangendo assim tanto o padrão antigo quanto o padrão Mercosul. O sistema também converte automaticamente a placa para letras maiúsculas no banco de dados.

RN06 - Bloqueio de Alteração de Histórico: Uma vez que a saída de um veículo é processada e o pagamento é confirmado pelo atendente, o registro muda permanentemente de "ATIVO" para "CONCLUIDO" no banco de dados. A partir desse momento, os dados de entrada, saída e valor pago se tornam imutáveis pela interface do sistema, servindo apenas para leitura no relatório histórico financeiro.

3.4 Casos de uso (descrição textual)

UC01 - Registrar Entrada de Veículo

- Ator Principal: Atendente.
- Resumo: O usuário registra a admissão de um veículo no pátio do estacionamento, gravando sua placa, categoria e o instante exato da entrada.
- Pré-condições: O sistema deve estar em execução e conectado ao banco de dados.
- Fluxo Principal (Caminho Feliz):
 1. O Atendente acessa a aba "Painel Operacional".
 2. O Atendente insere a placa do veículo no campo de texto correspondente.
 3. O Atendente seleciona a categoria do veículo na caixa de seleção ("CARRO" ou "MOTO").
 4. O Atendente clica no botão "Confirmar Entrada".
 5. O Sistema valida se a placa possui pelo menos 7 caracteres.
 6. O Sistema consulta o banco de dados para garantir que a placa não possui um registro "ATIVO".
 7. O Sistema converte a placa para letras maiúsculas e grava o registro no banco de dados com a data e hora do sistema (status "ATIVO").
 8. O Sistema exibe uma mensagem de sucesso ("Entrada registrada com sucesso!").
 9. O Sistema limpa o campo de digitação e atualiza a tabela de veículos estacionados em tempo real.

Fluxos de Exceção:

- FE01 - Placa Inválida: No passo 5, se a placa estiver vazia ou tiver menos de 7 caracteres, o sistema exibe uma mensagem de erro e aborta a operação.
- FE02 - Veículo já no Pátio: No passo 6, se o sistema identificar que a placa já consta como "ATIVA", ele exibe um aviso de duplicidade e aborta a operação para não sobrescrever dados.

UC02 - Registrar Saída e Pagamento (Finalizar Ticket)

- Ator Principal: Atendente.
- Resumo: O usuário busca um veículo estacionado pela placa, o sistema realiza o cálculo automático do valor devido, exibe o recibo e encerra o ciclo da estadia no banco de dados.
- Pré-condições: O veículo a ser baixado deve ter um registro prévio com status "ATIVO" no sistema.
- Fluxo Principal (Caminho Feliz):
 1. O Atendente, na aba "Painel Operacional", insere a placa do veículo no bloco "Registrar Saída / Pagamento".
 2. O Atendente clica no botão "Processar Saída".
 3. O Sistema consulta o banco de dados e recupera as informações ativas da placa informada.
 4. O Sistema captura a data e hora atual e calcula o tempo total de permanência em minutos.
 5. O Sistema identifica o tipo do veículo e aplica a regra de negócio tarifária correspondente (aplicação de polimorfismo, isenção por tolerância ou cálculo por hora cheia).
 6. O Sistema exibe um pop-up (Modal) contendo o "Recibo de Pagamento" com a Placa, Tipo, Tempo de Permanência e o Valor Total a ser cobrado, solicitando a confirmação do atendente.
 7. O Atendente recebe o dinheiro do cliente e clica em "Sim" (Confirmar).
 8. O Sistema atualiza o banco de dados, salvando a hora de saída, o valor pago e alterando o status do registro de "ATIVO" para "CONCLUIDO".
 9. O Sistema atualiza a lista de veículos no pátio, removendo o veículo em questão.

Fluxos de Exceção:

- FE01 - Placa em Branco: No passo 3, se o campo estiver vazio, o sistema exibe um erro de validação.

- FE02 - Veículo não encontrado: No passo 3, se a placa não constar como "ATIVA" no banco (digitada errada ou já liberada), o sistema informa que o veículo não foi encontrado e aborta a operação.
- FE03 - Cancelamento do Pagamento: No passo 7, se o cliente desistir ou houver algum problema no pagamento, o Atendente clica em "Não". O sistema fecha o recibo sem alterar o banco de dados e o veículo continua "ATIVO" no pátio.

UC03 - Consultar Histórico e Faturamento

- Ator Principal: Atendente / Administrador.
- Resumo: O usuário visualiza o histórico completo das movimentações financeiras e de rotatividade de veículos que já deixaram o estabelecimento.
- Pré-condições: O sistema deve estar executando com conexão ao banco de dados.
- Fluxo Principal:
 1. O usuário clica na aba "Histórico & Faturamento".
 2. O Sistema executa uma varredura no banco de dados buscando exclusivamente registros com o status "CONCLUIDO".
 3. O Sistema formata as datas de entrada e saída (DD/MM/AAAA HH:MM:SS) e converte os valores cobrados para o formato monetário padrão (R\$ 0,00).
 4. O Sistema preenche a tabela de histórico na tela, ordenando da saída mais recente para a mais antiga.
 5. (Opcional) A qualquer momento, o usuário pode clicar no botão "Atualizar Dados" no rodapé para forçar uma nova varredura e carregar dados recentes sem precisar reiniciar o programa.

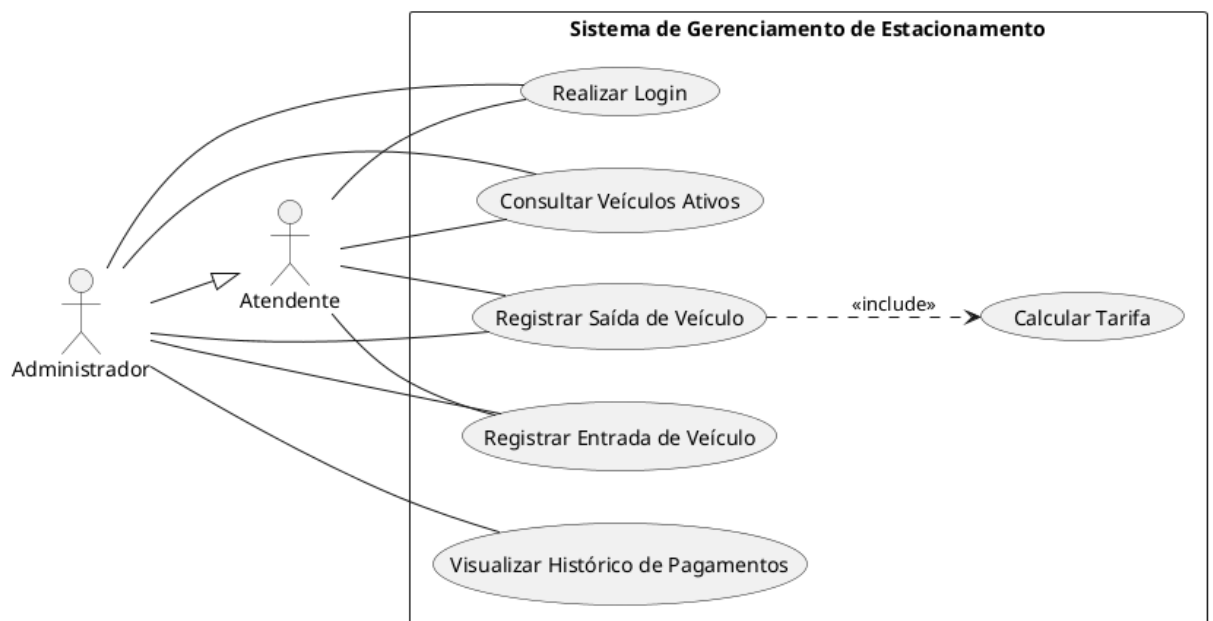
4. MODELAGEM DO SISTEMA

4.1 DFD (Diagrama de Fluxo de Dados)

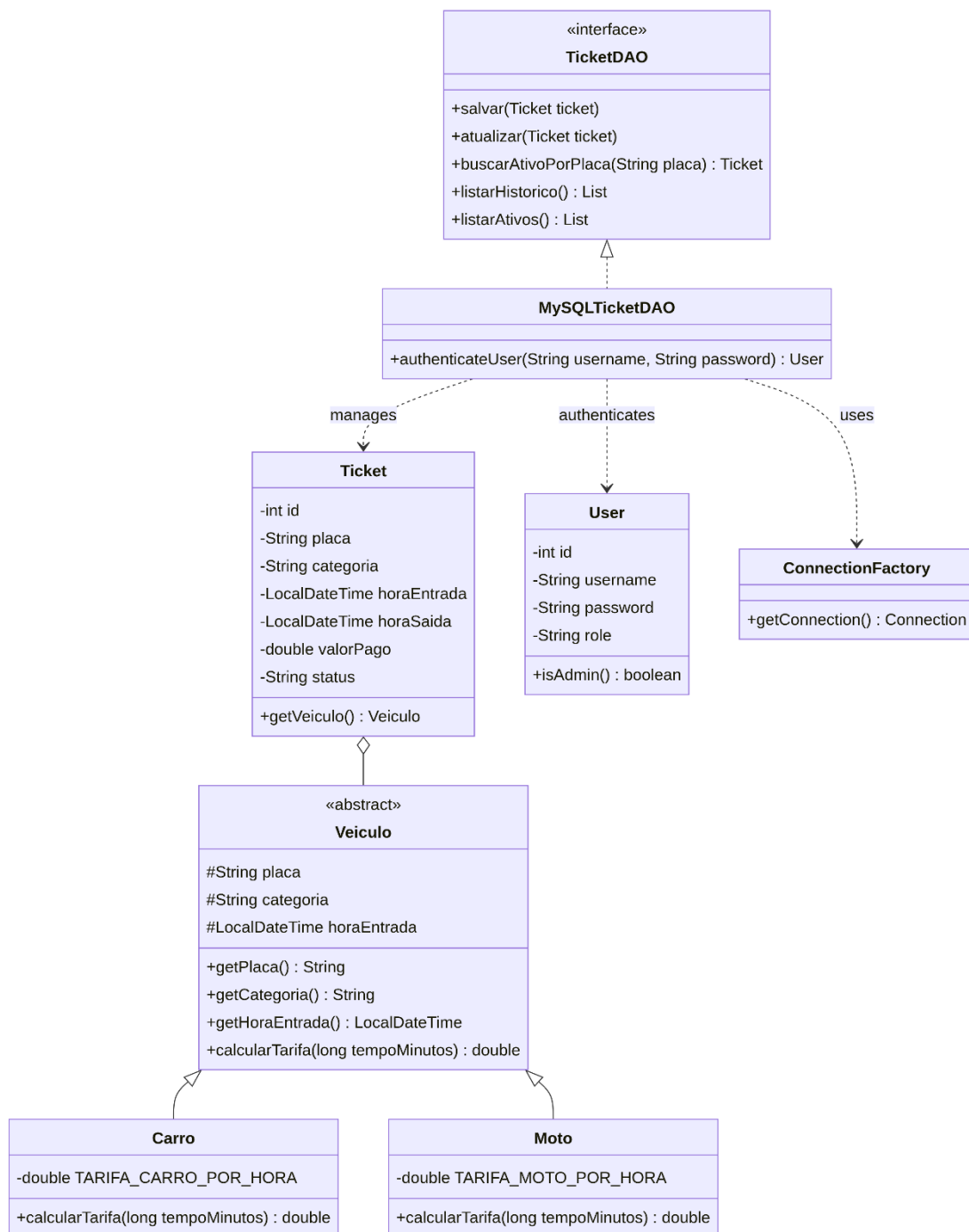


4.1 Diagramas UML

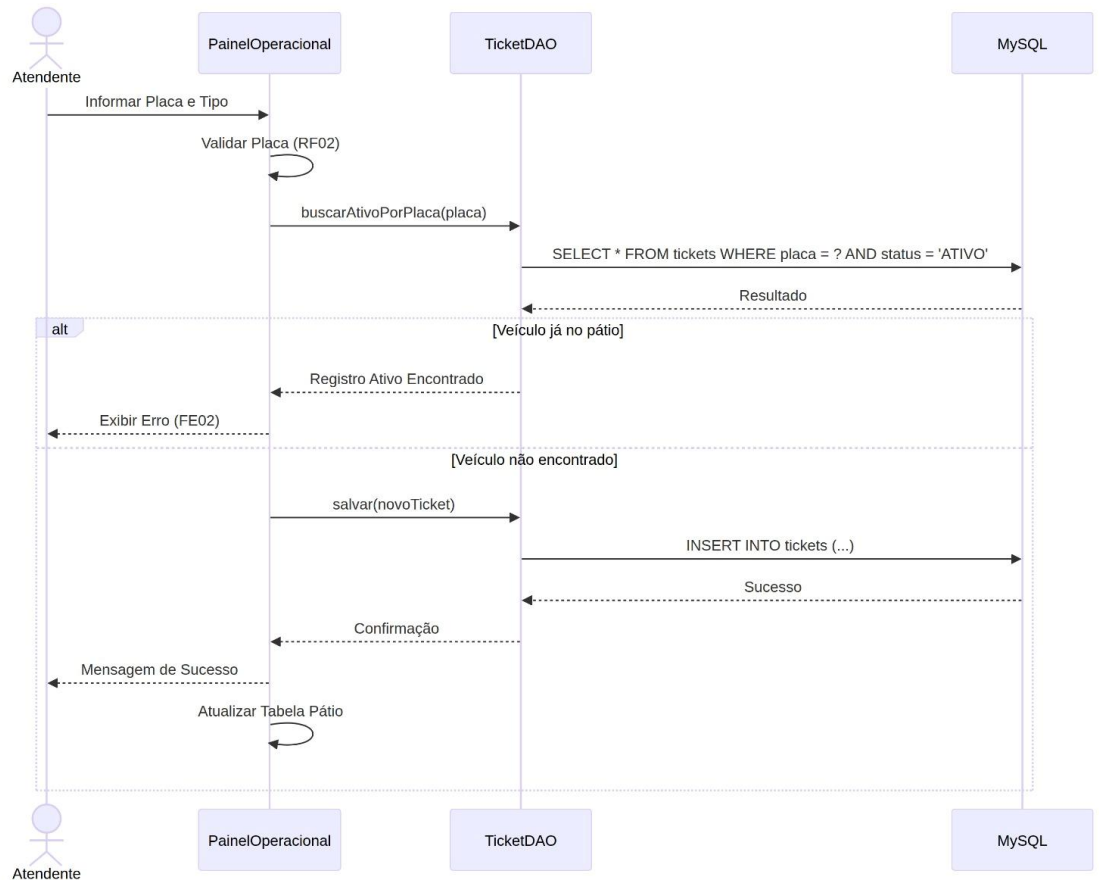
4..1.1 Diagrama de Casos de Uso



4.1.2 Diagrama de Classes



4.1.3 Diagrama de Sequência



5. PROTOTIPAÇÃO

5.1 Prototipação no Figma

<https://www.figma.com/make/moPKKXHCLOTOXd7kVocyPw/Classic-Java-Swing-UI-Design?t=jadLgo7Ee20wPKQ8-1>

5.2 Descrição das telas (Descrição das interfaces)

6. ARQUITETURA DO SISTEMA

6.1 Visão Geral da Arquitetura

O sistema foi concebido sob o modelo arquitetural de 3 Camadas (Três Camadas / 3-Tier Architecture) em um ambiente local (Cliente-Servidor standalone). A escolha desse padrão visa garantir a separação de responsabilidades (Separation of Concerns), facilitando a manutenção, a escalabilidade e o isolamento das regras de negócio em relação à interface do usuário.

A estrutura do projeto está logicamente dividida da seguinte forma:

- Camada de Apresentação (Presentation Layer / UI): Responsável exclusiva pela interação com o usuário final. Implementada através da classe principal (EstacionamentoApp), contém os componentes visuais em Java Swing (telas, botões, formulários e tabelas). Ela captura as ações do atendente e exibe os resultados, sem realizar cálculos de negócios diretamente.
- Camada de Lógica de Negócios (Business Logic Layer / Model): Representa o núcleo (core) do domínio do problema. Composta pelas entidades do sistema (Movimentacao e as classes abstratas e filhas Veiculo, Carro e Moto). É nesta camada que reside a aplicação de polimorfismo e as regras matemáticas de cálculo de tempo e tarifação por categoria.

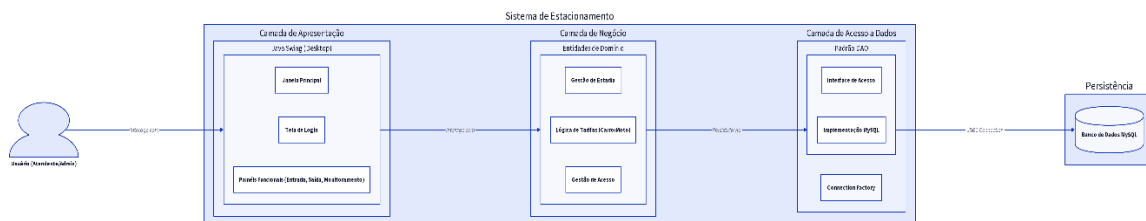
- Camada de Acesso a Dados (Data Access Layer / DAO): Encarregada da comunicação exclusiva com o Sistema de Gerenciamento de Banco de Dados. Implementada através da classe EstacionamentoDAO, ela isola toda a complexidade da linguagem SQL e da biblioteca JDBC (operações de Insert, Select, Update). A camada UI solicita os dados ao DAO, que por sua vez consulta o MySQL e retorna os objetos formatados para a tela.

6.2 Tecnologias utilizadas

O ecossistema tecnológico do projeto foi selecionado com foco em estabilidade, suporte nativo e adequação ao escopo de aplicações Desktop.

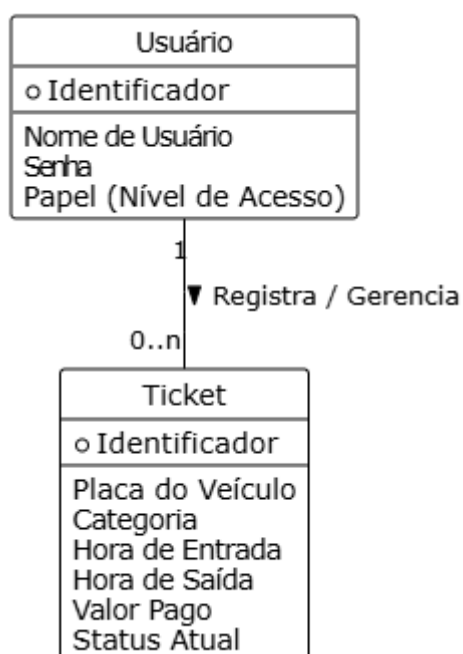
- Linguagem de Programação: Java (JDK 21). Escolhida pelo seu forte tipamento, portabilidade e excelente aderência aos conceitos de Programação Orientada a Objetos exigidos no projeto.
- Interface Gráfica: Java Swing. Biblioteca nativa padrão do Java (GUI Toolkit) utilizada para a construção e renderização das janelas e componentes da interface do usuário.
- Banco de Dados: MySQL. Sistema relacional rápido e leve, provido através da pilha de desenvolvimento XAMPP, ideal para persistência estruturada de dados de movimentação.
- Conectividade de Dados: API JDBC (Java Database Connectivity) com o driver MySQL Connector/J, permitindo a execução de instruções SQL e o mapeamento relacional diretamente pelo código Java.
- Ambiente de Desenvolvimento (IDE): Visual Studio Code (VS Code) auxiliado pelo pacote de extensões Extension Pack for Java.

6.3 Diagrama de arquitetura

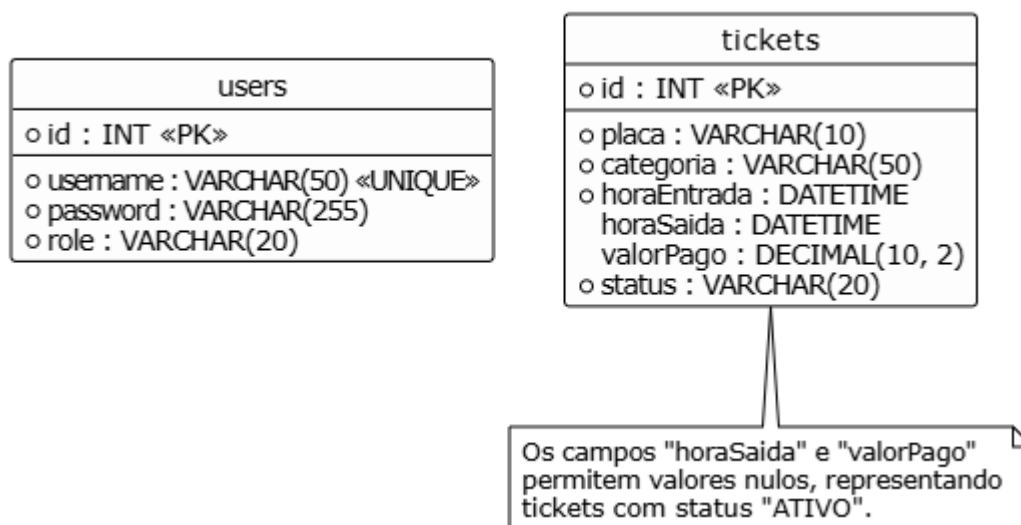


7. PROJETO DE BANCO DE DADOS

7.1 Modelo Conceitual



7.2 Modelo Lógico



7.3 Modelo Físico

8. DESENVOLVIMENTO / PROJETO FUNCIONAL

8.1 Código da aplicação (documentado)

O sistema foi implementado na linguagem Java, utilizando o pacote base com.estacionamento.app. A estrutura do código-fonte segue rigorosamente a arquitetura em três camadas (Apresentação, Lógica de Negócios e Acesso a Dados):

- Camada de Modelo (Model): Contém as classes que representam o domínio do problema. A classe abstrata `Veiculo` define os atributos base (placa, categoria, horaEntrada) e o método abstrato `calcularTarifa()`. As classes filhas `Carro` e `Moto` herdam de `Veiculo` e implementam o cálculo específico (R\$ 10,00 e R\$ 5,00 por hora, respectivamente), além de garantir a tolerância de 15 minutos. A classe `Ticket` gerencia o ciclo de vida da estadia, com status "ATIVO" ou "CONCLUIDO". A classe `User` define credenciais e papéis de acesso (ADMIN ou ATENDENTE).
- Camada de Acesso a Dados (DAO): Isolada no pacote dao, utiliza a interface `TicketDAO` para padronizar as operações de persistência. A classe `ConnectionFactory` centraliza a conexão com o banco de dados via JDBC. A classe principal de integração é a `MySQLTicketDAO`, que executa instruções SQL (INSERT, UPDATE, SELECT) de forma parametrizada com `PreparedStatement` para evitar injeções de SQL.

- Camada de Visão (View): Construída com a biblioteca Java Swing. O MainFrame atua como o contêiner principal , abrigando painéis modulares como EntradaPanel , SaidaPanel , MonitoramentoPanel e HistoricoPanel. O controle de acesso inicial é feito pela janela LoginFrame.

8.2 Scripts SQL (documentados)

O banco de dados relacional foi construído para persistir as informações operacionais e as credenciais de acesso. O script de inicialização Banco de dados.sql executa as seguintes rotinas estruturais:

```
CREATE DATABASE IF NOT EXISTS estacionamento;
USE estacionamento;

CREATE TABLE IF NOT EXISTS tickets (
    id INT AUTO_INCREMENT PRIMARY KEY,
    placa VARCHAR(10) NOT NULL,
    categoria VARCHAR(50) NOT NULL,
    horaEntrada DATETIME NOT NULL,
    horaSaida DATETIME,
    valorPago DECIMAL(10, 2),
    status VARCHAR(20) NOT NULL
);

CREATE TABLE IF NOT EXISTS users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    role VARCHAR(20) NOT NULL
);

INSERT IGNORE INTO users (username, password, role) VALUES ('admin', 'admin123', 'ADMIN');
INSERT IGNORE INTO users (username, password, role) VALUES ('atendente', '12345', 'ATENDENTE');
```

8.3 Padrões de codificação utilizados

O projeto aderiu a boas práticas de Engenharia de Software e Orientação a Objetos:

- Padrão DAO (Data Access Object): Utilizado para separar a lógica de acesso a dados da lógica de negócios, facilitando futuras manutenções.
- Polimorfismo e Herança: O método calcularTarifa(long tempoMinutos) na superclasse Veiculo é subscrito pelas classes filhas para aplicar a tarifação correta baseada no tipo de veículo instanciado dinamicamente via ticket.getVeiculo().
- Factory Method: Aplicado na criação de conexões com o banco de dados através da classe estática ConnectionFactory.
- Convenções da Linguagem: O código segue as nomenclaturas padrão do Java (CamelCase para métodos e variáveis, PascalCase para classes e UPPER_SNAKE_CASE para constantes globais como TARIFA_CARRO_POR_HORA).

9. PLANO DE TESTES (item importante adicionado)

9.1 Estratégia de testes

A validação do sistema focou em testes funcionais do tipo caixa-preta, executados manualmente através da Interface Gráfica, visando garantir que as regras de negócio fossem respeitadas desde a entrada do veículo até a persistência no banco de dados.

9.2 Casos de teste

- CT01 - Autenticação: Inserir credenciais válidas e inválidas no LoginFrame. (Critério de aceite: Acesso concedido apenas para 'admin' e 'atendente' cadastrados).
- CT02 - Regra de Placa (RN05): Tentar registrar entrada com placa em branco ou menor que 7 caracteres. (Critério de aceite: O sistema deve exibir erro e bloquear o INSERT).
- CT03 - Regra de Duplicidade (RN04): Registrar uma placa e tentar registrá-la novamente sem ter dado a saída. (Critério de aceite: O sistema deve bloquear a operação indicando que o veículo já está no pátio).
- CT04 - Tolerância (RN02): Registrar entrada de um veículo e dar saída com tempo inferior a 15 minutos. (Critério de aceite: O valor cobrado exibido no recibo deve ser estritamente R\$ 0,00).
- CT05 - Cobrança Polimórfica (RN01): Registrar entrada de Carro e de Moto, aguardar 1 hora e dar saída. (Critério de aceite: Sistema cobra R\$ 10,00 do carro e R\$ 5,00 da moto).

9.3 Resultados dos testes

Todos os casos de testes propostos apresentaram comportamento correspondente à especificação de requisitos e regras de negócios estabelecidas no capítulo 3. O polimorfismo para cálculo de estadia e as restrições de persistência via SQL atuaram sem gerar exceções (crashes) na máquina local.

10. CRONOGRAMA DO PROJETO

10.1 Etapas

1. Levantamento de Requisitos e Regras de Negócio.
2. Modelagem Arquitetural (DFD e Casos de Uso) e Prototipação UI.
3. Modelagem de Banco de Dados e estruturação do pacote Modelo (POO).
4. Implementação do pacote DAO e conexão JDBC.
5. Desenvolvimento das telas Java Swing e integração MVC.
6. Fase de Testes e Correção de Bugs.

10.2 Prazos

Semana 1: Análise de Requisitos e Modelagem (Tópicos 1, 2, 3 e 4).

Semana 2: Prototipação no Figma e Diagramas.

Semana 3: Codificação Java (POO, Views e Classes de Acesso a dados) e Banco MySQL (Tópico 6 e 8).

Semana 4: Testes de integração, documentação final e revisão (Tópicos 9 e 10).

10.3 Marcos (milestones)

Marco 1: Validação da documentação inicial e escopo aprovado.

Marco 2: Aprovação do design visual da aplicação e diagramas de classes.

Marco 3: Sistema conectado e persistindo dados corretamente no MySQL.

11. ORÇAMENTO

11.1 Custos estimados

Por se tratar de um projeto de viés acadêmico fundamentado em tecnologias open-source e bibliotecas nativas, não houveram custos diretos de licenciamento de software.

- Linguagem Java / JDK: Gratuito.
- SGBD MySQL (via XAMPP): Gratuito.
- IDE (Visual Studio Code): Gratuito.
- O único custo associável diz respeito ao esforço em horas de desenvolvimento (HH - Homem Hora) dos alunos alocados.

11.2 Recursos necessários

- Computador de desenvolvimento rodando sistema operacional Windows/Linux.
- Instalação do ambiente de desenvolvimento Java (JDK versão 21 ou superior).
- Pacote de serviços XAMPP para hospedagem local do servidor MySQL.

12. IMPLANTAÇÃO E MANUTENÇÃO (item adicionado)

1. Instalação do pacote XAMPP na máquina de destino para emular o servidor MySQL local na porta 3306.
2. Execução do script .sql (ver tópico 8.2) no phpMyAdmin para construção estrutural da base de dados.
3. Instalação do Java Runtime Environment (JRE) no sistema operacional.
4. Empacotamento do projeto via IDE e exportação do arquivo executável (.jar), que servirá como atalho na área de trabalho para os atendentes.
5. A manutenção preventiva foca em realizar backups programados (dumps) da tabela de 'tickets', garantindo a preservação do histórico de caixa.

13. REFERÊNCIAS BIBLIOGRÁFICAS

- ORACLE. Java SE 21 Documentation. Disponível em: <https://docs.oracle.com/en/java/javase/21/>. Acesso em: 2026.
- MYSQL. MySQL 8.0 Reference Manual. Disponível em: <https://dev.mysql.com/doc/refman/8.0/en/>. Acesso em: 2026.
- SOMMERVILLE, Ian. Engenharia de Software. 10ª ed. São Paulo: Pearson Education, 2018.

14. ANEXOS

14.1 Links úteis

- Download do XAMPP: https://www.apachefriends.org/pt_br/index.html
- Download do Java (JDK):
<https://www.oracle.com/java/technologies/downloads/>

14.2 Protótipos

O protótipo das telas desenhado vetorialmente pode ser acessado através da plataforma Figma:

<https://www.figma.com/make/moPKKXHCLOTOXd7kVocyPw/Classic-Java-Swing-UI-Design?t=jadLgo7Ee20wPKQ8-1>

15. LINK DO REPOSITÓRIO

15.1 GitHub do projeto

O versionamento do código base, histórico de commits e documentações podem ser analisados publicamente no repositório abaixo:

https://github.com/Durandai-F/projeto_estacionamento